

Week 2 Data analytics

Task 1:-

```
import numpy as np

arr = np.array([10, 20, 30, 40, 50, 60, 70, 80, 90, 100])

print("Array:", arr)

print("Shape:", arr.shape)

print("Size:", arr.size)

print("Data Type:", arr.dtype)

one_d = np.array([1, 2, 3, 4, 5])

print("\nOne-Dimensional Array:")

print(one_d)

two_d = np.array([[1, 2, 3], [4, 5, 6]])

print("\nTwo-Dimensional Array:")

print(two_d)
```

```
Array: [ 10  20  30  40  50  60  70  80  90 100]
Shape: (10,)
Size: 10
Data Type: int64

One-Dimensional Array:
[1 2 3 4 5]

Two-Dimensional Array:
[[1 2 3]
 [4 5 6]]
```

Task 2:-

```
import numpy as np
```

```
arr = np.array([10, 20, 30, 40, 50, 60, 70, 80, 90, 100])
```

```
print("Original Array:")
```

```
print(arr)
```

```
print("\nElement at index 0:", arr[0])
```

```
print("Element at index 4:", arr[4])
```

```
print("Element at index 7:", arr[7])
```

```
print("\nSliced Array (index 2 to 6):")
```

```
print(arr[2:7])
```

```
# Create a two-dimensional array
```

```
two_d = np.array([
```

```
    [1, 2, 3],
```

```
    [4, 5, 6],
```

```
    [7, 8, 9]
```

```
])
```

```
print("\nTwo-Dimensional Array:")
```

```
print(two_d)
```

```
print("\nFirst Row:")
```

```
print(two_d[0])
```

```
print("\nSecond Row:")
```

```
print(two_d[1])
```

```
print("\nFirst Column:")
```

```
print(two_d[:, 0])
```

```
print("\nSecond Column:")
```

```
print(two_d[:, 1])
```

```
print("\nElement at Row 2, Column 3:")
```

```
print(two_d[1, 2])
```

```
reshaped = arr.reshape(2, 5)
```

```
print("\nOriginal Array:")
```

```
print(arr)
```

```
print("\nReshaped Array (2 x 5):")
```

```
print(reshaped)
```

Task 3:-

```
import numpy as np
```

```
data1 = np.array([10, 20, 30, 40, 50])
```

```
data2 = np.array([2, 4, 5, 8, 10])
```

```
print("Dataset 1:", data1)
```

```
print("Dataset 2:", data2)
```

```
print("\n--- Mathematical Operations ---")
```

```
Original Array:
[ 10  20  30  40  50  60  70  80  90 100]

Element at index 0: 10
Element at index 4: 50
Element at index 7: 80

Sliced Array (index 2 to 6):
[30 40 50 60 70]

Two-Dimensional Array:
[[1 2 3]
 [4 5 6]
 [7 8 9]]

First Row:
[1 2 3]

Second Row:
[4 5 6]

First Column:
[1 4 7]

Second Column:
[2 5 8]

Element at Row 2, Column 3:
6
```

```
print("Addition:", data1 + data2)
```

```
print("Subtraction:", data1 - data2)
```

```
print("Multiplication:", data1 * data2)
```

```
print("Division:", data1 / data2)
```

```
print("\n--- Statistical Functions ---")
```

```
print("Mean:", np.mean(data1))
```

```
print("Median:", np.median(data1))
```

```
print("Minimum:", np.min(data1))
```

```
print("Maximum:", np.max(data1))
```

```
print("Standard Deviation:", np.std(data1))
```

```
print("Sum:", np.sum(data1))
```

```
Dataset 1: [10 20 30 40 50]
Dataset 2: [ 2  4  5  8 10]

--- Mathematical Operations ---
Addition: [12 24 35 48 60]
Subtraction: [ 8 16 25 32 40]
Multiplication: [ 20  80 150 320 500]
Division: [5. 5. 6. 5. 5.]

--- Statistical Functions ---
Mean: 30.0
Median: 30.0
Minimum: 10
Maximum: 50
Standard Deviation: 14.142135623730951
Sum: 150
```

Task 4:-

```
import pandas as pd
```

```
marks = pd.Series([85, 90, 78, 92, 88])
```

```
print("Pandas Series:")
```

```
print(marks)
```

```
data = {
```

```
    "Name": ["Aarav", "Priya", "Rahul", "Sneha", "Aman"],
```

```

    "Age": [20, 21, 19, 20, 22],
    "Marks": [85, 90, 78, 92, 88]
}

```

```

df = pd.DataFrame(data)

print("\nOriginal DataFrame:")
print(df)

print("\nColumn Names:")
print(df.columns)

print("\nIndex:")
print(df.index)

```

```

df["Grade"] = ["B", "A", "C", "A", "B"]

print("\nUpdated DataFrame:")
print(df)

```

Task 5:-

```

import pandas as pd

df = pd.read_csv("students.csv")

print("First 5 Rows:")

print(df.head())

```

```

Pandas Series:
0    85
1    90
2    78
3    92
4    88
dtype: int64

Original DataFrame:
   Name  Age  Marks
0  Aarav   20    85
1  Priya   21    90
2  Rahul   19    78
3  Sneha   20    92
4   Aman   22    88

Column Names:
Index(['Name', 'Age', 'Marks'], dtype='object')

Index:
RangeIndex(start=0, stop=5, step=1)

Updated DataFrame:
   Name  Age  Marks  Grade
0  Aarav   20    85     B
1  Priya   21    90     A
2  Rahul   19    78     C
3  Sneha   20    92     A
4   Aman   22    88     B

```

```
print("\nLast 5 Rows:")
```

```
print(df.tail())
```

```
print("\nNumber of Rows and Columns:")
```

```
print(df.shape)
```

```
print("\nColumn Names:")
```

```
print(df.columns)
```

```
print("\nData Types:")
```

```
print(df.dtypes)
```

```
print("\nDataset Information:")
```

```
df.info()
```

```
print("\nStatistical Description:")
```

```
print(df.describe())
```

task 6:-

```
import pandas as pd
```

```
data = {
```

```
    "Name": ["Aarav", "Priya", "Rahul", "Sneha", "Aman", "Riya"],
```

```
    "Age": [20, 21, 19, 20, 22, 19],
```

```
    "Marks": [85, 92, 68, 95, 78, 88],
```

```
Name, Age, Marks, Grade
```

```
Aarav, 20, 85, B
```

```
Priya, 21, 90, A
```

```
Rahul, 19, 78, C
```

```
Sneha, 20, 92, A
```

```
Aman, 22, 88, B
```

```
Riya, 19, 75, C
```

```
Karan, 21, 95, A
```

```
Neha, 20, 82, B
```

```

"City": ["Delhi", "Mumbai", "Delhi", "Pune", "Mumbai", "Delhi"]
}

```

```

df = pd.DataFrame(data)

```

```

print("Original DataFrame:")

```

```

print(df)

```

```

print("\nSelected Columns (Name and Marks):")

```

```

print(df[["Name", "Marks"]])

```

```

print("\nSelected Rows (first 3 rows):")

```

```

print(df.iloc[0:3])

```

```

print("\nStudents with Marks greater than 80:")

```

```

print(df[df["Marks"] > 80])

```

```

print("\nStudents with Marks greater than 80 and Age less
      than 21:")

```

```

print(df[(df["Marks"] > 80) & (df["Age"] < 21)])

```

```

print("\nStudents sorted by Marks (Ascending):")

```

```

print(df.sort_values(by="Marks", ascending=True))

```

```

print("\nStudents sorted by Marks (Descending):")

```

```

print(df.sort_values(by="Marks", ascending=False))

```

Original DataFrame:

	Name	Age	Marks	City
0	Aarav	20	85	Delhi
1	Priya	21	92	Mumbai
2	Rahul	19	68	Delhi
3	Sneha	20	95	Pune
4	Aman	22	78	Mumbai
5	Riya	19	88	Delhi

Selected Columns (Name and Marks):

	Name	Marks
0	Aarav	85
1	Priya	92
2	Rahul	68
3	Sneha	95
4	Aman	78
5	Riya	88

Selected Rows (first 3 rows):

	Name	Age	Marks	City
0	Aarav	20	85	Delhi
1	Priya	21	92	Mumbai
2	Rahul	19	68	Delhi

Students with Marks greater than 80:

	Name	Age	Marks	City
0	Aarav	20	85	Delhi
1	Priya	21	92	Mumbai
3	Sneha	20	95	Pune
5	Riya	19	88	Delhi

Task 7:-

```
import pandas as pd
```

```
import numpy as np
```

```
data = {
```

```
    "Name": ["Aarav", "Priya", "Rahul", "Sneha", "Aman", "Riya"],
```

```
    "Age": [20, 21, np.nan, 20, 22, np.nan],
```

```
    "Marks": [85, np.nan, 68, 95, np.nan, 88],
```

```
    "City": ["Delhi", "Mumbai", "Delhi", np.nan, "Mumbai", "Delhi"]
```

```
}
```

```
df = pd.DataFrame(data)
```

```
print("Dataset Before Handling Missing Values:")
```

```
print(df)
```

```
print("\nMissing Values (True/False):")
```

```
print(df.isnull())
```

```
print("\nMissing Values using isna():")
```

```
print(df.isna())
```

```
print("\nCount of Missing Values in Each Column:")
```

```
print(df.isnull().sum())
```

```
df_removed = df.dropna()
```

```
print("\nDataset After Removing Rows with Missing Values:")
```

```
print(df_removed)
```

```
df_filled = df.copy()
```

```
df_filled["Age"] = df_filled["Age"].fillna(df_filled["Age"].mean())
```

```
df_filled["Marks"] = df_filled["Marks"].fillna(df_filled["Marks"].mean())
```

```
df_filled["City"] = df_filled["City"].fillna(df_filled["City"].mode()[0])
```

```
print("\nDataset After Filling Missing Values:")
```

```
print(df_filled)
```

	Name	Age	Marks	City
0	Aarav	20.00	85.0	Delhi
1	Priya	21.00	84.0	Mumbai
2	Rahul	20.75	68.0	Delhi
3	Sneha	20.00	95.0	Delhi
4	Aman	22.00	84.0	Mumbai
5	Riya	20.75	88.0	Delhi

Task 8:-

```
import pandas as pd

students = pd.DataFrame({
    "Student_ID": [101, 102, 103, 104, 105],
    "Name": ["Aarav", "Priya", "Rahul", "Sneha", "Aman"],
    "Department": ["IT", "CS", "IT", "CS", "IT"]
})

marks = pd.DataFrame({
    "Student_ID": [101, 102, 103, 104, 105],
    "Marks": [85, 92, 78, 95, 88],
    "Attendance": [90, 95, 80, 92, 85]
})

print("Students DataFrame:")
print(students)

print("\nMarks DataFrame:")
print(marks)

merged_df = pd.merge(students, marks, on="Student_ID")

print("\n--- Merged DataFrame ---")
print(merged_df)
```

```
students_part1 = pd.DataFrame({
```

```
    "Student_ID": [101, 102, 103],
```

```
    "Name": ["Aarav", "Priya", "Rahul"],
```

```
    "Marks": [85, 92, 78]
```

```
})
```

```
students_part2 = pd.DataFrame({
```

```
    "Student_ID": [104, 105, 106],
```

```
    "Name": ["Sneha", "Aman", "Riya"],
```

```
    "Marks": [95, 88, 82]
```

```
})
```

```
concatenated_df = pd.concat(
```

```
    [students_part1, students_part2],
```

```
    ignore_index=True
```

```
)
```

```
print("\n--- Concatenated DataFrame ---")
```

```
print(concatenated_df)
```

```
grouped = merged_df.groupby("Department")
```

```
print("\n--- GroupBy: Mean Marks by Department ---")
```

```
print(grouped["Marks"].mean())
```

```
aggregation = merged_df.groupby("Department")["Marks"].agg(  
    ["sum", "mean", "count", "min", "max"]  
)
```

```
print("\n--- Aggregate Values by Department ---")  
print(aggregation)
```

```
pivot = pd.pivot_table(  
    merged_df,  
    values="Marks",  
    index="Department",  
    columns="Name",  
    aggfunc="mean"  
)
```

```
print("\n--- Pivot Table ---")  
print(pivot)
```

Students DataFrame:

	Student_ID	Name	Department
0	101	Aarav	IT
1	102	Priya	CS
2	103	Rahul	IT
3	104	Sneha	CS
4	105	Aman	IT

Marks DataFrame:

	Student_ID	Marks	Attendance
0	101	85	90
1	102	92	95
2	103	78	80
3	104	95	92
4	105	88	85

--- Merged DataFrame ---

	Student_ID	Name	Department	Marks	Attendance
0	101	Aarav	IT	85	90
1	102	Priya	CS	92	95
2	103	Rahul	IT	78	80
3	104	Sneha	CS	95	92
4	105	Aman	IT	88	85

--- Concatenated DataFrame ---

	Student_ID	Name	Marks
0	101	Aarav	85
1	102	Priya	92
2	103	Rahul	78
3	104	Sneha	95
4	105	Aman	88

Task 10:-

```
import numpy as np
```

```
import pandas as pd
```

```
data = {  
    "Student_ID": [101, 102, 103, 104, 105, 106, 107, 108],  
    "Name": ["Aarav", "Priya", "Rahul", "Sneha", "Aman", "Riya", "Karan", "Neha"],  
    "Gender": ["Male", "Female", "Male", "Female", "Male", "Female", "Male", "Female"],  
    "Department": ["IT", "CS", "IT", "CS", "IT", "CS", "IT", "CS"],  
    "Age": [20, 21, np.nan, 20, 22, 19, 21, np.nan],  
    "Marks": [85, 92, 68, 95, np.nan, 88, 78, 90],  
    "Attendance": [90, 95, 80, 92, 85, np.nan, 88, 94]  
}
```

```
df = pd.DataFrame(data)
```

```
print("Original Dataset:")
```

```
print(df)
```

```
print("\nFirst 5 Rows:")
```

```
print(df.head())
```

```
print("\nLast 5 Rows:")
```

```
print(df.tail())
```

```
print("\nShape of Dataset:")
```

```
print(df.shape)
```

```
print("\nColumn Names:")
```

```
print(df.columns)
```

```
print("\nData Types:")
```

```
print(df.dtypes)
```

```
print("\nDataset Information:")
```

```
df.info()
```

```
print("\nStatistical Summary:")
```

```
print(df.describe())
```

```
print("\nMissing Values in Each Column:")
```

```
print(df.isnull().sum())
```

```
df["Age"] = df["Age"].fillna(df["Age"].mean())
```

```
df["Marks"] = df["Marks"].fillna(df["Marks"].mean())
```

```
df["Attendance"] = df["Attendance"].fillna(df["Attendance"].mean())
```

```
print("\nDataset After Handling Missing Values:")
```

```
print(df)
```

```
print("\nMissing Values After Cleaning:")
```

```
print(df.isnull().sum())
```

```
print("\nSelected Columns - Name, Department and Marks:")
```

```
print(df[["Name", "Department", "Marks"]])
```

```
print("\nStudents Scoring 85 or More:")
```

```
print(df[df["Marks"] >= 85])
```

```
print("\nIT Students with Marks 80 or More:")
```

```
print(df[(df["Department"] == "IT") & (df["Marks"] >= 80)])
```

```
print("\nStudents Sorted by Marks - Highest to Lowest:")
```

```
sorted_df = df.sort_values(by="Marks", ascending=False)
```

```
print(sorted_df)
```

```
group_analysis = df.groupby("Department").agg(
```

```
    Average_Marks=("Marks", "mean"),
```

```
    Maximum_Marks=("Marks", "max"),
```

```
    Minimum_Marks=("Marks", "min"),
```

```
    Average_Attendance=("Attendance", "mean"),
```

```
    Student_Count=("Student_ID", "count")
```

```
)
```

```
print("\nGroupBy Analysis by Department:")
```

```
print(group_analysis)
```

```
pivot_table = pd.pivot_table(
```

```
    df,
```

```
    values="Marks",
```

```
    index="Department",
```

```
    columns="Gender",
```

```
    aggfunc="mean"
```

```
)
```

```
print("\nPivot Table - Average Marks by Department and Gender:")
```

```
print(pivot_table)
```

```
average_marks = df["Marks"].mean()
```

```
highest_marks = df["Marks"].max()
```

```
lowest_marks = df["Marks"].min()
```

```
top_student = df.loc[df["Marks"].idxmax(), "Name"]
```

```
department_average = df.groupby("Department")["Marks"].mean()
```

```
highest_department = department_average.idxmax()
```

```
print("\n===== USEFUL INSIGHTS =====")
```

```
print("Overall Average Marks:", round(average_marks, 2))
```

```
print("Highest Marks:", highest_marks)
```

```
print("Lowest Marks:", lowest_marks)
```

```
print("Top Performing Student:", top_student)
```

```
print("Department with Highest Average Marks:", highest_department)
```

```
print("=====")
```

```
output_file = "processed_student_performance.csv"
```

```
df.to_csv(output_file, index=False)
```

```
print("\nCleaned dataset exported successfully!")
```

```
print("File Name:", output_file)
```

```
verified_df = pd.read_csv(output_file)
```

```
print("\nVerified Exported Dataset:")
```

```
print(verified_df)
```

```
print("\nRows:", verified_df.shape[0])
```

```
print("Columns:", verified_df.shape[1])
```

```
print("\nMissing Values in Exported Dataset:")
```

```
print(verified_df.isnull().sum())
```

```
print("\nCSV file is ready for submission.")
```

```
Cleaned dataset exported successfully!
```

```
File Name: processed_student_performance.csv
```

```
Verified Exported Dataset:
```

	Student_ID	Name	Gender	Department	Age	Marks	Attendance
0	101	Aarav	Male	IT	20.0	85.000000	90.000000
1	102	Priya	Female	CS	21.0	92.000000	95.000000
2	103	Rahul	Male	IT	20.5	68.000000	80.000000
3	104	Sneha	Female	CS	20.0	95.000000	92.000000
4	105	Aman	Male	IT	22.0	85.142857	85.000000
5	106	Riya	Female	CS	19.0	88.000000	89.142857
6	107	Karan	Male	IT	21.0	78.000000	88.000000
7	108	Neha	Female	CS	20.5	90.000000	94.000000

```
Rows: 8
```

```
Columns: 7
```

```
Missing Values in Exported Dataset:
```

```
Student_ID    0
```

```
Name          0
```

```
Gender        0
```

```
Department    0
```

```
Age           0
```

```
Marks         0
```

```
Attendance    0
```

```
dtype: int64
```